



Security Assessment Final Report



Safenet (Part 2)

August 2026

Prepared for Safe

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	4
Threat and Security Overview.....	5
Assets.....	5
Actors.....	6
Roles and Trust Boundaries.....	7
Assumptions.....	7
Attack Surface.....	8
FROST hashing, identifiers, and nonces.....	8
Signature and signature-share verification.....	9
Secp256k1 point arithmetic.....	10
Documentation and integration.....	10
Known Issues.....	11
Findings Summary.....	12
Severity Matrix.....	12
Detailed Findings.....	13
Informational Severity Issues.....	14
I-01: Multiple comments and NatSpec entries are inaccurate.....	14
I-02: The nonce helper can expose signing shares if used on-chain.....	16
I-03: Signature verification failure reverts with the same error for many different scenarios.....	18
I-04: _expandMessageXmd can wrap its one-byte block counter for unsupported output lengths.....	20
I-05: Identity additions return aliased memory references.....	22
I-06: _divmod returns zero instead of rejecting a zero divisor.....	24
I-07: mulmuladd() accepts identity points that it can never verify.....	26
I-08: Participant identifiers are not explicitly required to be nonzero.....	28
I-09: Cancun-only opcode (MCOPY) and the ModExp precompile are required.....	30
Disclaimer.....	31
About Certora.....	31

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safenet	https://github.com/safe-research/safenet	Audited commit: dbfd8fd Fix commit: 3c0d939	Solidity

Project Overview

This document describes the security review of **Safenet (Part 2)**. The work was undertaken from **August 19th, 2026** to **August 25th, 2026**.

The following contracts are included in the scope:

contracts/src/libraries/FROST.sol

contracts/src/libraries/Secp256k1.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Safenet adds a second authorization layer to owner-signed Safe transactions. **SafenetGuard** is installed as a Safe transaction guard and checks each **execTransaction** before the Safe executes it. A transaction is permitted when it carries a valid Safenet FROST threshold-signature attestation, when it matches a matured time-delayed announcement, or when it is an auto-allowed call used to manage that announcement mechanism.

The guard runs on the Safe's target chain. It does not call Gnosis Chain or execute transactions through **FROSTCoordinator**. Instead, it keeps a local set of trusted Safenet epoch keys, reconstructs the attested EIP-712 message, verifies the FROST signature, and returns control to the Safe, which performs the actual call or delegatecall.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

This threat model covers only the contracts reviewed during the second part of the Safenet audit:

- `contracts/src/libraries/FROST.sol`
- `contracts/src/libraries/Secp256k1.sol`

The libraries are stateless cryptographic building blocks. `FROST.sol` implements hashing, participant identifiers, nonce derivation, binding factors, challenges, and verification of FROST signatures and signature shares. `Secp256k1.sol` implements point validation, serialization, equality, affine addition, and the `ecrecover`-based verification of $zG - eP = R$.

Calling contracts are considered only to establish reachability. The Guard, Consensus, coordinator, DKG and signing state machines, epoch rollover, Safe authorization, validators, and operational key management are outside the scope.

Verdicts describe the implementation state at the time of this review:

- **Impossible** means an enforced invariant prevents the threat.
- **Mitigated** means the threat is possible in principle but is sufficiently constrained.
- **By design** means the behavior is intentional and creates a caller or deployment obligation.
- **Audit finding** means a concrete issue identified during this review and tracked in the current audit report. "Audit finding" does not imply prior unawareness or lack of acknowledgment by the project team.
- **Out of scope** means the relevant control belongs to a component outside the two libraries.

Assets

- **Final-signature soundness** — `FROST.verify()` must accept only signatures satisfying $zG = R + cY$ for the supplied group key and message.

- **Signature-share equation soundness** — `FROST.verifyShare()` must correctly check the supplied participant equation without accepting malformed points or scalars.
- **FROST signing shares and nonces** — participant secret shares and nonce material must remain confidential and nonces must never be reused.
- **Public-key and point integrity** — public keys, commitments, and intermediate points must be valid secp256k1 elements wherever the protocol requires a non-identity element.
- **Transcript correctness and interoperability** — domain-separated hashes, point encodings, participant identifiers, challenges, and scalar reductions must match the supported FROST ciphersuite and off-chain implementation.
- **Verification availability** — valid supported signatures and shares must not become unverifiable because of hidden key restrictions or incompatible EVM features.
- **Arithmetic and memory integrity** — curve operations must produce the correct value and must not unexpectedly corrupt or alias caller-owned memory.
- **Diagnostic integrity** — callers and monitoring systems should be able to distinguish an invalid witness from a permanently unsupported input or precompile failure.
- **Documentation accuracy** — NatSpec and comments must accurately describe cryptographic inputs, limitations, terminology, and deployment requirements.

Actors

- **Untrusted caller or integrator** — may expose an internal library function through a public entry point and may supply arbitrary messages, points, scalars, commitments, identifiers, or randomness.
- **Malicious FROST participant** — controls its own commitments and submitted signature share and may attempt to provide identity points, malformed elements, or inconsistent data.
- **Honest signing participant** — holds a secret signing share and is expected to generate fresh secret nonce material off-chain.
- **Honest signer threshold** — can produce a valid aggregate signature; fewer than the threshold must not be able to forge one.

- **Off-chain FROST implementation** — derives identifiers, runs DKG and signing rounds, generates nonces, and exchanges values with the Solidity libraries.
- **EVM execution environment** — supports the required EVM revision and the `ecrecover`, SHA-256, and ModExp precompiles on which the implementation relies.

Roles and Trust Boundaries

- `FROST.sol` is responsible for constructing its documented hash transcripts and correctly checking the full-signature and signature-share equations supplied to it.
- `Secp256k1.sol` is responsible for validating coordinates, representing the identity consistently, adding curve points, serializing non-identity points, and enforcing the `mulmuladd()` witness equation.
- The integrating contract is responsible for choosing the message and its application-level domain separation, admitting suitable public keys, and deciding what authorization follows from a successful verification.
- Signer software is responsible for CSPRNG use, nonce secrecy, nonce uniqueness, secret-share storage, and protocol-round correctness.
- The deployment process is responsible for selecting a compatible EVM revision and ensuring that all required precompiles are available.

Assumptions

- The secp256k1 discrete-logarithm problem is infeasible at the intended security level.
- SHA-256 provides the properties required by the FROST Fiat-Shamir construction.
- Honest signers use 32 fresh bytes from a CSPRNG for nonce generation and never reuse randomness or derived nonces with the same signing share.
- Secret signing shares and nonce-generation inputs never appear in transaction calldata or pass through an untrusted RPC endpoint.
- Group keys and participant verification shares admitted by an integration are valid non-identity points with $x < N$, reflecting the `ecrecover` limitation described in [I-O3](#).

- Full signatures and commitment shares do not use the identity element, as required by RFC 9591 serialization.
- The current `_expandMessageXmd()` call continues to request exactly 48 bytes; callers cannot select an unsupported output length.
- The existing callers of `Secp256k1.add()` do not mutate an identity-branch result through its aliased memory reference.
- The only callers of `_divmod()` remain the two guarded branches in `add()`, whose divisors are nonzero for valid canonical curve points.
- The target EVM has activated Cancun/EIP-5656 and provides `ecrecover (0x01)`, `SHA-256 (0x02)`, and `ModExp (0x05)`.

Attack Surface

FROST hashing, identifiers, and nonces

- **Nonce derivation is invoked in a transaction with a real signing share — Verdict: Audit finding (I-02).** `nonce(bytes32 random, uint256 secret)` performs only the deterministic H3 step. If exposed through a transaction, both inputs are public in calldata: the signing share is disclosed directly and the nonce is recomputable. The function also cannot enforce CSPRNG quality or nonce non-reuse. It currently has no production caller.
- **A reused nonce allows recovery of a participant's signing share — Verdict: Out of scope.** The library has no nonce state and cannot reserve or burn nonce material. Prevention belongs to signer software; [I-02](#) requires the helper's documentation or location to make this boundary explicit.
- `identifier()` returns zero — Verdict: Audit finding (I-08). Hash-to-field reduction can theoretically produce zero, but the function does not reject it. A zero identifier reveals the DKG polynomial's constant term to that participant and defeats the intended threshold. The Rust implementation rejects zero, the Solidity helpers have no production caller, and the probability is approximately $1/N$.
- `_expandMessageXmd()` is called with more than 8,160 output bytes — Verdict: Audit finding (I-04). RFC 9380 permits at most 255 SHA-256 blocks. The implementation permits lengths below 32,768, so its one-byte block counter wraps for larger requests. The private

helper's only caller requests 48 bytes, making this a standards-compliance and future-maintenance issue rather than a current exploit.

- **The XMD helper is treated as a general RFC 9380 implementation — Verdict: Audit finding (I-01).** Its NatSpec overstates its generality: it permits excessive block counts and supports only the contract's custom, at-most-31-byte DST representation.
- **The hash-to-field documentation causes an integrator to confuse `m` with `count` — Verdict: Audit finding (I-01).** Both values are one here, so behavior is unaffected, but the comment uses the wrong RFC 9380 terminology.

Signature and signature-share verification

- **A noncanonical signature scalar is accepted — Verdict: Impossible.** `verify()` and `verifyShare()` require their `z` scalar to be less than the secp256k1 group order.
- **An off-curve or out-of-field affine point is accepted as a normal curve point — Verdict: Impossible.** `_satisfiesCurveEquation()` checks both coordinate bounds and the secp256k1 equation. `_unpack()` additionally permits only the explicit `(0,0)` identity sentinel.
- `mulmuladd()` accepts the identity as `p` or `r` — Verdict: Audit finding (I-07). `_unpack()` admits `(0,0)` because point addition needs an identity, but `mulmuladd()` cannot verify either identity argument. Identity `p` produces invalid ECDSA `r = 0`; identity `r` is negated into malformed `(0,P)`. Both fail closed with `InvalidMulMulAddWitness`. Full signature verification rejects identity points during serialization, while `verifyShare()` can reach this path through `share.r`.
- **A valid public key with `x >= N` is treated as an ordinary bad signature — Verdict: Audit finding (I-03).** Such a point can satisfy the secp256k1 curve equation because coordinates are bounded by `P`, but it cannot be used as the ECDSA `r` value required by the `ecrecover` reduction. Retrying with another signature cannot repair this permanent key limitation.
- **An effective challenge multiplier of zero is treated as an ordinary bad signature — Verdict: Audit finding (I-03).** The constructed ECDSA `s` is zero and `ecrecover` rejects it, even if the underlying equation is mathematically valid. This case is cryptographically negligible but demonstrates the same diagnostic ambiguity.

- **A failed or malformed `ecrecover` call is indistinguishable from a witness mismatch** — Verdict: Audit finding ([I-03](#)). The assembly masks failure to zero and all unsuccessful paths revert with `InvalidMulMulAddWitness`. Callers cannot distinguish routine rejection, a permanently unsupported key, an effective multiplier of zero, and an exceptional precompile failure.

Secp256k1 point arithmetic

- **Adding the identity returns a memory reference that aliases an input** — Verdict: Audit finding ([I-05](#)). The identity branches return `p` or `q` directly. Because `Point memory` is a reference type, mutating the result also mutates that operand within the call frame. Current callers do not perform such a mutation.
- **A zero divisor makes `_divmod()` return zero instead of reverting** — Verdict: Audit finding ([I-06](#)). The helper computes $0^{(p-2)} \bmod p = 0$, the precompile reports success, and the function returns the valid-looking value zero. The issue is unreachable through current `add()` branches because their denominators are nonzero for valid finite secp256k1 points, but the helper does not enforce its own mathematical precondition.
- **Point addition can pass a zero divisor through its current branches** — Verdict: Impossible for valid inputs. The addition branch requires different x-coordinates, and a finite secp256k1 point cannot have `y = 0` in the doubling branch. The identity is handled before division.

Documentation and integration

- **Inaccurate comments and NatSpec misdescribe the code** — Verdict: Audit finding ([I-01](#)). The report identifies incorrect descriptions of participant commitments, nonce generation, aggregate commitment parameters, RFC `m`, XMD generality, inverse-point branching, `ModExp` gas, unsupported-key probability, grammar, and typographical errors. These do not change execution but can mislead cryptographic integrators and maintainers.
- **Verification is deployed to a pre-Cancun EVM** — Verdict: Audit finding ([I-09](#)). `_expandMessageXmd()` uses `MCOPY`, introduced by Cancun/EIP-5656. On a pre-Cancun EVM, signature verification reaches an invalid opcode.

- **Point addition is deployed where the ModExp precompile is unavailable — Verdict: Audit finding (I-09).** `Secp256k1.add()` relies on ModExp for field inversion, in addition to the verification path's `ecrecover` and SHA-256 requirements. Deployment documentation and scripts must require all applicable EVM features.

Known Issues

There was a single known issue before the start of the audit, which was communicated to the auditors and documented in the following comment in `mulmuladd`:

```
Additionally, it only supports points `P` with x-coordinates that are elements in Fn.  
In practice (assuming uniform distribution of x-coordinates), ...
```

This means that public keys with an x-coordinate satisfying $N \leq x < P$ are not supported by this verification method. The issue is acknowledged, and the protocol accepts the risk due to its low likelihood.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	–	–	–
Informational	9	9	9
Total	9	9	9

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	Multiple comments and NatSpec entries are inaccurate	Informational	Fixed
I-02	The nonce helper can expose signing shares if used on-chain	Informational	Fixed
I-03	Signature verification failure reverts with the same error for many different scenarios	Informational	Fixed
I-04	_expandMessageXmd can wrap its one-byte block counter for unsupported output lengths	Informational	Fixed
I-05	Identity additions return aliased memory references	Informational	Fixed
I-06	_divmod returns zero instead of rejecting a zero divisor	Informational	Fixed
I-07	mulmuladd() accepts identity points that it can never verify	Informational	Fixed
I-08	Participant identifiers are not explicitly required to be nonzero	Informational	Fixed
I-09	Cancun-only opcode (MCOPY) and the ModExp precompile are required	Informational	Fixed

Informational Severity Issues

I-01: Multiple comments and NatSpec entries are inaccurate

Files:

contracts/src/libraries/FROST.sol
contracts/src/libraries/Secp256k1.sol

Description:

Several comments and NatSpec entries in `FROST.sol` and `Secp256k1.sol` incorrectly describe the implementation.

FROST.sol

- `Commitment` is described as a group commitment, but it contains one participant's hiding and binding nonce commitments.
- `nonce()` says that it generates a random nonce and implements `nonce_generate`. It only performs the deterministic hashing step using randomness supplied by the caller; it does not generate that randomness.
- The `r` parameter of both `challenge()` and `verifyShare()` is described as a participant commitment, but it is the aggregate group commitment used to compute the challenge.
- `_hashToField()` describes `m` as the number of elements generated. In RFC 9380, `m` is the field extension degree; `count` is the number of elements generated. Both values are `1` here.
- `_expandMessageXmd()` claims to implement RFC 9380's general `expand_message_xmd` function, but `assert(len < 0x8000)` permits more than the RFC maximum of 255 output blocks. The helper also supports only the contract's custom, at-most-31-byte DST representation.

Secp256k1.sol

- The comment for `N` says "All scalar arithmetic are performed"; this should be "All scalar arithmetic is performed."
- In `add()`, the final branch is said to occur when $P = -P$. It actually occurs when $Q = -P$.

- The `add()` documentation states that the `ModExp` call costs approximately 200 gas. For these operands, the precompile charge is 1,360 gas under EIP-2565 and 4,080 gas under EIP-7883, excluding call overhead.
- The `mulmuladd()` documentation says “e time the point” instead of “e times the point.”
- The `mulmuladd()` caveat gives the wrong frequency for unsupported public keys. It requires $p.x < N$; a valid point with $N \leq p.x < P$ is unsupported. This occurs with probability approximately 3.73×10^{-39} , or once in 2.68×10^{38} uniformly generated points, rather than once in 3.7×10^{39} points.

These errors do not change contract behavior, but inaccurate cryptographic documentation can mislead integrators, maintainers, and auditors.

Recommendations:

Correct the comments and NatSpec to match the implementation and use RFC terminology accurately.

Customer Response:

Fixed in [PR#853](#)

Fix Review:

Fix confirmed.

I-02: The nonce helper can expose signing shares if used on-chain

Files:

contracts/src/libraries/FROST.sol

Description:

`FROST.nonce(bytes32 random, uint256 secret)` implements only the deterministic hashing step of RFC 9591's `nonce_generate`:

```
return _h3(abi.encodePacked(random, secret));
```

The function expects the caller to supply both the randomness and the participant's secret signing share. If it is ever exposed and called in a transaction, both values will be public in calldata. Anyone could then recompute the nonce, and the signing share itself would already be compromised.

The function also does not enforce the RFC's operational requirements: `random` must contain 32 fresh bytes from a cryptographically secure random-number generator and must never be reused with the same secret. Nonce reuse can enable recovery of a participant's signing share.

There is no current exploit because the function is `internal` and has no production caller. It is only used in tests. However, its NatSpec describes it as generating a random nonce without warning future integrators that it is unsafe to invoke in a transaction or with secrets sent through an untrusted RPC endpoint.

Recommendations:

Prefer removing the helper from the on-chain library and performing nonce generation entirely within the signer. Otherwise, add an explicit `@dev` warning that:

- the helper implements only the H3 hashing portion of `nonce_generate`;
- it must only be used off-chain or through a trusted local `eth_call`, never in a transaction;
- `random` must be 32 fresh CSPRNG bytes
- randomness and derived nonces must never be reused with the same signing share.

See RFC 9591 [Section 4.1](#) for nonce generation and [Section 7.3](#) for nonce-reuse attacks. [Section 7.1](#) separately covers side-channel protections for secret-dependent operations.

Customer Response:

Fixed in [PR#854](#)

Fix Review:

Fix confirmed.

I-O3: Signature verification failure reverts with the same error for many different scenarios

Files:

contracts/src/libraries/Secp256k1.sol

Description:

`Secp256k1.mulmuladd()` reports every unsuccessful equation check with `InvalidMulMulAddWitness`. This mixes several materially different conditions:

- the signature or signature share is genuinely invalid;
- the public key is a valid curve point but has $x \geq N$, so it can never be processed by `ecrecover`.
- the effective challenge multiplier is zero, causing the constructed ECDSA `s` value to be zero and `ecrecover` to fail.
- for any other reason, the static call to `ecrecover` returns false or a return data of size different than 32 bytes.

The assembly masks an unsuccessful or malformed `ecrecover` result to zero and ultimately reaches the same check used for an ordinary equation mismatch:

```
require(valid, InvalidMulMulAddWitness());
```

Consequently, callers cannot distinguish a routine signature rejection from a permanently unusable key.

The $x \geq N$ case therefore appears as a routine rejection even though retrying with another signature cannot help. The `e == 0` case is cryptographically negligible, but it demonstrates the same ambiguity for a mathematically valid equation that the `ecrecover` reduction cannot process.

Recommendations:

Prevent the permanent failure in the first place by requiring $x < N$ whenever group keys and participant verification shares are accepted.

Additionally, separate unsupported-input and precompile-failure conditions from an invalid witness. For example:

- revert with a dedicated error when the public key has $x \geq N$;
- revert with a dedicated error when the challenge e is zero;
- explicitly validate the `ecrecover` call's success and return-data length; and
- reserve `InvalidMulMulAddWitness` for a successful recovery whose result does not match the supplied witness.

This allows callers, monitoring infrastructure, and incident-response tooling to distinguish invalid signatures from permanent key-configuration failures and exceptional limitations of the verification primitive.

Customer Response:

Fixed in [PR#855](#)

Fix Review:

Fix confirmed.

I-04: `_expandMessageXmd` can wrap its one-byte block counter for unsupported output lengths

Files:

contracts/src/libraries/FROST.sol

Description:

RFC 9380 limits `expand_message_xmd` to 255 output blocks. With SHA-256, each block is 32 bytes, so the maximum supported output length is 8,160 bytes.

`_expandMessageXmd` allows any `len` below 32,768 instead:

```
function _expandMessageXmd(bytes memory message, bytes32 dst, uint256 len)
    private
    view
    returns (bytes memory uniform)
{
    // ...
    assert(len < 0x8000);
```

The function writes the block number with `mstore8`, which keeps only the lowest byte:

```
for { let i := 2 } gt(len, 0x20) {
    i := add(i, 1)
    len := sub(len, 32)
} {
    let uPtr := add(uniform, shl(5, i))
    mstore(bPtr, xor(b0, mload(sub(uPtr, 0x20))))
    mstore8(iPtr, i)
    _sha256(bPtr, bLen, uPtr)
    ...
}
```

If `len` is greater than 8,160, the function needs more than 255 blocks. The counter then wraps: block 256 is encoded as `0x00`, block 257 as `0x01`, and so on. The result does not match a conforming RFC 9380 implementation.

This cannot affect the current protocol. `_expandMessageXmd` is private, and its only call site always requests 48 bytes:

```
function _hashToField(bytes memory message, bytes32 dst) private view
returns(uint256 e) {
    // ...
    bytes memory uniform = _expandMessageXmd(message, dst, 48);
```

For 48 bytes, the function uses only two blocks, so the counter cannot wrap. External callers cannot choose `len`. This is an Informational standards-compliance and future-maintenance issue, not a reachable security issue in the current code.

Recommendations:

Reject output lengths above 8,160 bytes. For example, replace the current assertion with `assert(len <= 8160)`. This keeps the block count within the one-byte RFC 9380 limit.

Customer Response:

Fixed in [PR#856](#)

Fix Review:

Fix confirmed.

I-O5: Identity additions return aliased memory references

Files:

contracts/src/libraries/Secp256k1.sol

Description:

`Secp256k1.add()` returns an input point directly when either operand is the point at infinity:

```
...
if (px | py == 0) {
    return q;
} else if (qx | qy == 0) {
    return p;
}
...
```

Because `Point memory` is a Solidity memory reference type and `add()` is an internal function, these branches return the existing memory reference rather than copying the coordinates into the fresh return value. The behavior therefore differs from ordinary additions, which write the result into `r`.

A caller that treats the result as an independent value can unintentionally modify the corresponding operand:

```
Secp256k1.Point memory sum = Secp256k1.add(identity, point);
sum.x = newX; // Also changes point.x in this call frame.
```

The alias exists only in memory within the same call frame. Assigning the result to storage or returning it through an external call performs a copy.

The current code does not modify the value returned by `add()`, but this issue can cause bugs if the library is reused or the current callers of the function are refactored.

Recommendations:

Give `add()` consistent value-like behavior by copying identity operands into the named return value:

```
...
if (px | py == 0) {
    r.x = qx;
    r.y = qy;
    return r;
} else if (qx | qy == 0) {
    r.x = px;
    r.y = py;
    return r;
}
...
```

Customer Response:

Fixed in [PR#857](#)

Fix Review:

Fix confirmed.

I-06: `_divmod` returns zero instead of rejecting a zero divisor

Files:

contracts/src/libraries/Secp256k1.sol

Description:

`_divmod` computes field division as $x * y^{(p-2)} \bmod p$. This formula only defines an inverse when y is not zero.

The helper does not check $y \neq 0$. If y is zero, the `ModExp` precompile returns zero and reports success. The following `mulmod` also returns zero, and `assert(success)` does not reject the invalid divisor:

```
        mstore(add(ptr, 0x60), y)
        mstore(add(ptr, 0x80), sub(p, 2))
        mstore(add(ptr, 0xa0), p)
        success := and(eq(returndatasize(), 0x20), staticcall(gas(), 0x5,
ptr, 0xc0, 0x00, 0x20))

        // Compute `x / y = x * inv(y)`.
        result := mulmod(x, mload(0x00), p)
    }
    assert(success);
```

This turns an undefined operation into the valid-looking field element zero. A future caller could continue with the wrong result instead of failing at the invalid input.

The issue cannot occur in the current code. `_divmod` is private and has only two call sites, both in `Secp256k1.add`:

```
    } else if (px != qx) {
        // Point addition, compute the slope through `P` and `Q`:
        //      λ = (Qy - Py) / (Qx - Px)
        unchecked {
            l = _divmod(addmod(qy, P - py, P), addmod(qx, P - px, P), P);
        }
    } else if (py == qy) {
```



```
// Point doubling, compute the slope of the tangent at `P`:  
//       $\lambda = (3 \cdot Px^2 + a) / (2 \cdot Py)$   
//       $= (3 \cdot Px^2) / (2 \cdot Py)$   
// Noting that `a = 0` for the `secp256k1` curve.  
unchecked {  
    1 = _divmod(mulmod(3, mulmod(px, px, P), P), addmod(py, py, P), P);  
}
```

The point-addition branch requires $px \neq qx$, so $(qx - px) \bmod P$ cannot be zero for canonical coordinates. The point-doubling branch uses $2 * py \bmod P$. A valid finite point on secp256k1 cannot have $py = 0$. The function also handles the $(0,0)$ infinity sentinel before these branches. Therefore, both current divisors are always non-zero.

No external input can trigger the invalid division today. This is an informational defensive programming issue.

Recommendations:

Reject a zero divisor inside `_divmod`, for example, add `require(y != 0, InvalidDivisor())` before calling `ModExp`.

Customer Response:

Fixed in [PR#864](#)

Fix Review:

Fix confirmed.

I-07: mulmuladd() accepts identity points that it can never verify

Files:

contracts/src/libraries/Secp256k1.sol

Description:

`Secp256k1` represents the point at infinity as $(0,0)$. The private `_unpack()` function accepts this value because `add()` needs an identity element when accumulating curve points, as used in `FROSTSignatureShares.register()`, which sums signature shares starting from a zero-initialized accumulator:

```
valid := or(iszero(or(x, y)), valid)
```

`mulmuladd()` uses `_unpack()` for **both** of its point arguments, so it accepts $(0,0)$ as the point `p` and as the expected result `r`. Neither is verifiable.

For `r`, the function negates the witness before comparing it with the point recovered through `ecrecover`:

```
mstore(0x00, rx)
mstore(0x20, sub(P, ry))
```

For a normal point, $(rx, P - ry)$ correctly represents $-r$. For the identity representation $(0,0)$ this produces $(0,P)$, which is neither the identity nor a valid secp256k1 point. $y = P$ fails the coordinate range check that every real point satisfies.

For `p`, an identity value yields $px = 0$, which the `ecrecover` precompile rejects as an invalid `r` signature component.

In both cases the call reverts with `InvalidMulMulAddWitness`, so there is no safety impact. The defect is that the library accepts inputs it can never validate, and in the `r` case silently constructs a malformed encoding on the way to rejecting them.

This is not a missing capability. A relation of the form:

```
z * G - e * P = 0
```

is mathematically valid but protocol-invalid: [RFC 9591 Section 6.5](#) requires `SerializeElement` to reject the identity element, so a FROST signature or commitment share equal to the point at infinity is invalid by definition. Correcting the negation to produce $(0,0)$ would not help either, because the `ecrecover` precompile cannot return the point at infinity, so the underlying verification technique does not support this result. The correct behavior is to reject the input early, not to support it.

Reachability

`FROST.verify()` cannot reach this path: `signature.r` is serialized inside `challenge()` before `mulmuladd()` runs, and `serialize()` rejects the identity.

`FROST.verifyShare()` can. Its `share.r` argument is never serialized, so a participant can submit `share.r = (0,0)` directly because it neither derives nor authenticates `share.r` against the participant's nonce commitments..

Recommendations:

Explicitly disallow the identity value for both point arguments at the start of `mulmuladd()`:

```
requireNonZero(p);
requireNonZero(r);
```

This changes only which error is raised (the call reverts today with `InvalidMulMulAddWitness` and would revert with `NotOnCurve`) but it fails fast on an input that can never succeed and removes the malformed $(0,P)$ construction. Applying it to the library leaves all existing tests passing.

Customer Response:

Fixed in [PR#865](#)

Fix Review:

Fix confirmed.

I-08: Participant identifiers are not explicitly required to be nonzero

Files:

contracts/src/libraries/FROST.sol

Description:

FROST represents each participant with a number called an identifier. These identifiers must be nonzero.

`FROST.identifier()` derives an identifier by hashing the participant's address and reducing the result modulo the secp256k1 group order:

```
function identifier(address participant) internal view returns (uint256 id) {  
    return _hid(abi.encodePacked(participant));  
}
```

Reduction modulo the group order can produce zero, but the function does not reject that result.

A zero identifier is not just a nonstandard encoding. During FROST key generation, participants receive a share of a secret polynomial evaluated at their identifier. A participant whose identifier is zero receives the polynomial evaluated at zero, which is the group secret itself. This would defeat the intended threshold because that participant could sign without the others.

The off-chain Rust implementation avoids this outcome because `Identifier::derive()` rejects zero. The Solidity and Rust implementations therefore handle this edge case differently: Solidity returns zero, while Rust returns an error. In the current Rust wrapper, that error would trigger an `expect()` panic.

Some Solidity paths also reject zero indirectly. In particular, `_encodeCommitments()` initializes `last` to zero and requires every identifier to be greater than `last`, so a zero identifier cannot be included in that commitment list. This does not make the behavior consistent, because `identifier()` itself and `keyGenChallenge()` still accept the value.

There is no practical attack or impact through the current on-chain call paths. No contract under `contracts/src` reaches the Solidity identifier helpers, Rust rejects zero, and the probability for a particular address is approximately $1 / N$, or 2^{-256} . Even considering the entire 160-bit Ethereum address space, finding such an address is infeasible.

Recommendations:

Reject zero directly in `identifier()` so that every Solidity caller receives the same guarantee:

```
id = _hid(abi.encodePacked(participant));  
require(id != 0, InvalidIdentifier());
```

Customer Response:

Fixed in [PR#866](#)

Fix Review:

Fix confirmed.

I-09: Cancun-only opcode (MCOPY) and the ModExp precompile are required

Files:

contracts/src/libraries/FROST.sol
contracts/src/libraries/Secp256k1.sol

Description:

The protocol documentation says that Safenet attestations can be verified on any EVM-compatible chain. It lists only the `ecrecover` (`0x01`) and `SHA-256` (`0x02`) precompiles as requirements.

`_expandMessageXmd` uses `MCOPY` (EIP-5656, Cancun), and it sits on the signature-verification path (`FROST.verify` → `challenge` → `_h2` → `_hashToField` → `_expandMessageXmd`) that `SafenetGuard` executes on every attested Safe transaction, on every chain a Safe can be deployed to. `Secp256k1.add` additionally depends on the `ModExp` (`0x05`) precompile. So a deployment to a chain that has not activated Cancun will revert with an invalid opcode on every attestation check, permanently bricking the guard (which cannot be upgraded — the domain separator and epoch forest are immutable/append-only).

Recommendations:

Explicitly document Cancun (EIP-5656) plus the `ModExp` precompile as hard deployment prerequisites and gate the deployment scripts accordingly.

Customer Response:

Fixed in [PR#867](#)

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.